TED UNIVERSITY

CMPE 492 – Senior Project II

Spring 2025

AKKE – Low Level Design Report

(SMART COMMAND AND CONTROL GLOVE)

Webpage: https://ilter-akke.github.io/website/

Name Surname Department ID

Abdullah ESİN (CMPE)

Berk ÇAKMAK (CMPE)

Ömer Efe DİKİCİ (CMPE)

Şevval KURTULMUŞ (EEE)

Supervisors: Ali BERKOL and Hüseyin Uğur YILDIZ

Jury Members: Hakkı Gökhan İLK and Mehmet Evren COŞKUN

Table of Contents

1. Introduction *3*

1.1. Architectural Revision from HLD to LLD **3**

2. Design Principles & Creativity *4*

2.1. Design Principles **4**

2.2. Creativity and Novelty **4**

3. Engineering Standards *5*

4. Object Design Trade-offs *6*

5. Interface Documentation Guidelines *7*

6. Packages & Component Interfaces *8*

6.1. Data Acquisition Package **8**

6.2. Processing & Core Control Package **9**

6.3. Communication Interfaces (Glove & Receiver Packages) **11**

6.4 Class Diagram **12**

7. Interface Contracts and Data Formats *13*

8. Boundary Conditions & Error Handling *14*

9. Glossary *15*

10. References *16*

1. Introduction

The purpose of this Low-Level Design (LLD) report is to bridge the gap between the high-level Design (HLD) application objects defined in the CMPE 491 analysis phase and the off-the-shelf hardware components of the AKKE system. By refining the high-level architecture into a detailed low-level model, this document specifies the precise hardware interfaces and peripherals such as ADC, I2C, UART, PWM, and GPIO required for the ESP32-WROOM-32 microcontroller. This LLD refines the CMPE 491 HLD into implementation-ready firmware and hardware interfaces that will serve as the direct blueprint for the testing and deployment phases.

1.1. Architectural Revision from HLD to LLD

During the transition from high-level analysis to low-level implementation, a major architectural revision was made. The original HLD proposed an ESP32-S3 microcontroller and transmitted analog audio continuously over an RF module. In this LLD, the architecture has been revised to utilize the **ESP32-WROOM-32** and **ESP-NOW digital triggering**. This revision was executed to significantly reduce communication latency, prevent analog audio jamming, lower power consumption, and distribute the processing load by moving audio playback to the local storage of the receiver unit.

2. Design Principles & Creativity

2.1. Design Principles

The low-level design of the AKKE system is governed by several core software and hardware engineering principles:

- **Modularity and Separation of Concerns:** The system architecture effectively separates data acquisition (flex and IMU sensors), processing (TinyML), and communication (ESP-NOW). This modular approach ensures that the machine learning model or sensor calibration routines can be updated independently without disrupting the underlying hardware drivers.
- **Edge Computing (Local Processing):** The practice of processing data fully local on the device (the ESP32-WROOM-32) rather than relying on a centralized cloud, ensuring minimal latency and offline operation.
- **Hardware Abstraction:** The design employs a Hardware Abstraction Layer (HAL) to interface with the MPU6050 via I2C and the flex sensors via ADC. This decouples the raw physical voltage and register readings from the high-level gesture recognition logic.

2.2. Creativity and Novelty

The creativity of this project lies in merging wearable sensor technology with Edge AI to eliminate operational vulnerabilities. During the design phase, three alternatives were evaluated for communication:

- **Alternative 1:** Analog RF audio streaming (High power, susceptible to noise/jamming).
- **Alternative 2:** Standard Wi-Fi/Bluetooth (High latency, pairing overhead).
- **Alternative 3:** ESP-NOW digital command packets.

The novel choice of implementing **Alternative 3 (ESP-NOW)** combined with TinyML on a resource-constrained ESP32-WROOM-32 represents a highly creative and robust solution. This ensures field personnel can execute synchronized audio commands without exposing their position through continuous radio emissions.

3. Engineering Standards

To ensure the reliability, maintainability, and interoperability of the AKKE system, the low-level design and hardware-software integration are designed in accordance with several recognized industry engineering standards:

- **UML (Unified Modeling Language) - ISO/IEC 19505:** The system's architectural documentation, including class interfaces, sequence flows, and state machine behaviors, is designed in accordance with UML standards. This provides a standardized visual blueprint for the object-oriented design phase.
- **IEEE 802.11 (Wireless LANs):** While the system utilizes Espressif's ESP-NOW protocol for low-latency peer-to-peer communication, this protocol operates on the 2.4 GHz ISM band and is more robust against interference, drawing upon action frame mechanisms defined under the IEEE 802.11 standard [7].
- **I2C (Inter-Integrated Circuit):** Communication between the ESP32-WROOM-32 microcontroller and the MPU6050 IMU sensor utilizes the I2C bus specification as supported by the sensor's hardware architecture [6]. The system operates at a 400 kHz clock speed for rapid data acquisition.
- **UART (Universal Asynchronous Receiver-Transmitter):** Communication with the external audio module (e.g., DFPlayer Mini) on the receiver side is handled via UART, a de facto industry standard for asynchronous serial communication, ensuring reliable transmission of hex-based audio commands.
- **C++ Standard:** The embedded firmware for the ESP32-WROOM-32, including the hardware abstraction layer, sensor drivers, and TinyML integration, is developed using standard C++ paradigms, ensuring efficient memory management and object-oriented encapsulation [4], [5].

4. Object Design Trade-offs

- **ESP-NOW Digital Triggering vs. Analog RF Audio Broadcast:** The design shifted to ESP-NOW. *Trade-off:* This significantly reduces communication latency and power consumption at the cost of requiring a secondary microcontroller (ESP32) and local audio storage on the receiver side.
- **Microcontroller Selection (ESP32-S3 vs. ESP32-WROOM-32):** *Trade-off:* The WROOM-32 was chosen over the S3 due to immediate component availability, cost-effectiveness, and mature community library support, while still providing sufficient SRAM and dual-core performance for TinyML inference.
- **TinyML on Edge vs. Cloud Processing:** *Trade-off:* Running the model locally constrains the complexity and size of the neural network but enables full offline functionality.

5. Interface Documentation Guidelines

To close the gap between application objects and off-the-shelf components, each class interface in this report is documented with specific attributes:

- **Hardware Mapping:** The exact peripheral or GPIO pin utilized.
- **Responsibility:** The core function of the class.
- **Method Signatures:** Exact data types for inputs and returns to ensure contract stability.

Preconditions, Postconditions, and Constraints: Specifies required initial states, expected outcomes, timing constraints, and failure cases where applicable.

6. Packages & Component Interfaces

6.1. Data Acquisition Package

Class: FlexSensor

- **Hardware Mapping:** ADC Channels (GPIO 25, 33, 32, 35, 34).
- **Methods:**
  - std::array<uint16_t, 5> readRawFlex()
    - *Precondition:* ADC must be initialized to 12-bit resolution.
    - *Postcondition:* Returns an array of 5 raw digital values (0-4095).
    - *Timing:* Execution < 1ms.
  - std::array<float, 5> getNormalizedData()
    - *Precondition:* calibrateSensors() must have been successfully executed at least once.
    - *Postcondition:* Returns calibrated values mapped between 0.0 and 1.0.
  - void calibrateSensors()
    - **Postcondition:** Calculates and stores the baseline offset values for all 5 flex sensors.

Class: IMUSensor

- **Hardware Mapping:** I2C (SDA: GPIO 21, SCL: GPIO 22).
- **Methods:**
  - IMUData readIMUData()
    - *Precondition:* I2C bus initialized at 400kHz; device 0x68 is responsive.
    - **Postcondition:** Returns populated IMUData struct (int16_t ax, ay, az, gx, gy, gz; bool isValid).
    - **Failure Case:** If I2C read fails, isValid is set to false and last known good values are returned.

6.2. Processing & Core Control Package

Class: GestureClassifier (TinyML Core)

- **Responsibility:** Processes the feature vector to predict the gesture.
- **Methods:**
  - uint8_t predictGesture(const FeatureVector& data)
    - *Inputs:* A standardized FeatureVector derived from the 2-second sensor window, containing the normalized flex and IMU features required by the TinyML model.
    - *Outputs:* Returns the predicted command_id (0x01 to 0x05). Returns 0xFF if the confidence score is below the 75% threshold (acting as the Decision Gate).
    - *Timing Constraints:* Inference must complete in < 50ms to maintain real-time responsiveness.

Class: GloveController (Main State Machine)

- **Responsibility:** Orchestrates the system states (Idle, Calibrating, Collecting, Recognizing, Transmitting) and manages exception states (Error, Low Battery).
- **Methods:**
  - **void initializeSystem():** Initializes serial communications, sensor HALs, and ESP-NOW protocols.
  - **void runLoop():** The main execution thread evaluating state transitions based on sensor timing and interrupts.
  - **void processRecognitionCycle():** Reads normalized sensor data and triggers the TinyML inference model.
  - **void transitionToState(uint8_t newState):** Safely handles the entry and exit conditions between the defined system states.
  - **void handleTimeout():** Resets the system to the Idle state if sensor buffering exceeds the predefined time limit.

The primary operational states are Idle, Calibrating, Collecting, Recognizing, and Transmitting; Error and LowBattery are treated as exception states.

The control behavior of the glove is governed by a finite-state machine managed by the GloveController. Figure 1 illustrates the primary operational states and the exception paths used in the low-level design.

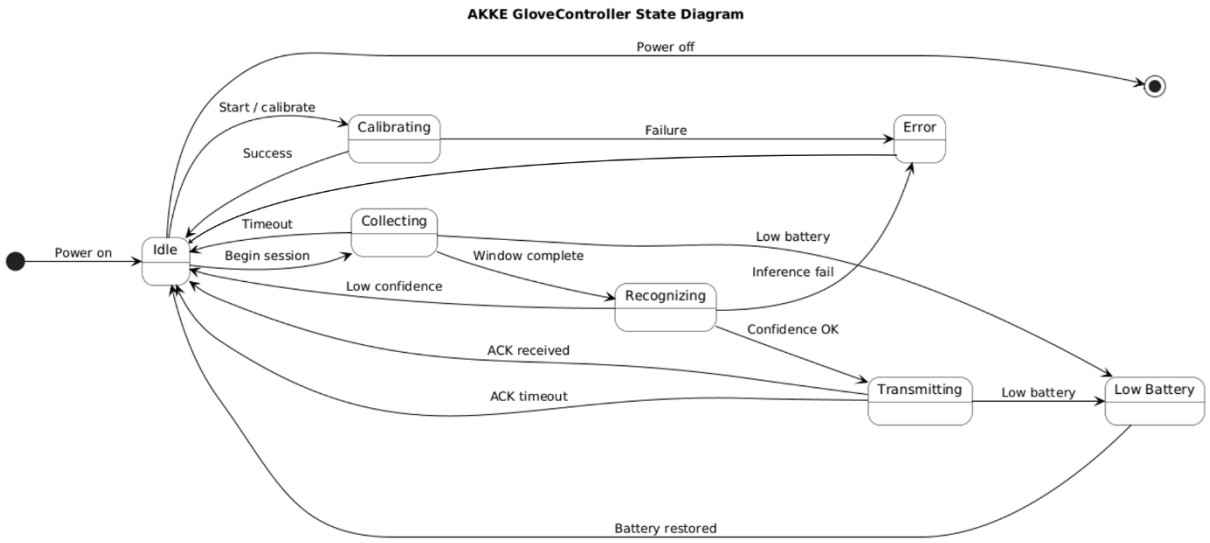

Figure 1 UML state diagram of the AKKE GloveController showing operational and exception states.

As shown in Figure 1, the nominal control flow progresses from Idle to Collecting, Recognizing, and Transmitting, while Calibrating, Error, and Low Battery are modeled as exception or initialization states that interrupt or precede normal execution when required.

6.3. Communication Interfaces (Glove & Receiver Packages)

Class: ESPNowSender (Glove Side)

- **Hardware Mapping:** ESP32 Internal 2.4GHz Wi-Fi Antenna.
- **Methods:**
  - bool sendCommandPayload(uint8_t cmd_id)
    - *Inputs:* 1-byte payload (cmd_id).
    - *Postcondition:* Returns true if MAC-layer ACK is received from the paired Receiver Unit.
    - *Retry Policy:* Retries up to 3 times with a 10ms timeout per attempt.

Class: AudioEngine (Receiver Side)

- **Hardware Mapping:** UART (TX/RX) operating at 9600 baud.
- **Methods:**
  - void playCommand(uint8_t track_id)
    - *Precondition:* track_id must be within the valid range (1-5). Invalid or reserved payloads such as 0xFF are filtered before audio playback is triggered.
    - *Postcondition:* Triggers the DFPlayer Mini via hex instruction.

6.4 Class Diagram

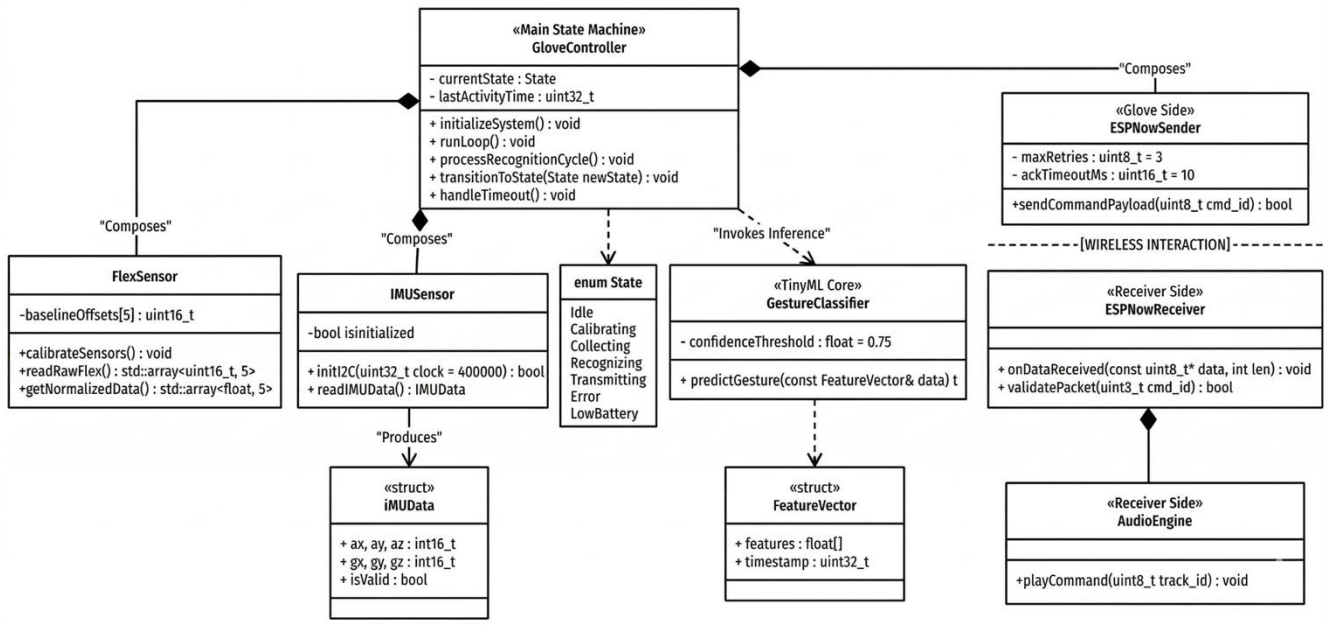

Figure 2 UML class diagram of the AKKE system showing the main software components, data structures, and communication relationships between the glove-side and receiver-side modules.

7. Interface Contracts and Data Formats

To ensure strict subsystem integration, the following data formats and contracts are established:

- **Sensor Sampling Rate:** Sensors are sampled at **200 Hz** (every 5 ms).
- **Window Size:** The ML model evaluates a sliding window of **2 seconds** (400 samples per axis).
- **ESP-NOW Payload:** The transmitted packet is strictly **1 Byte**. In future iterations, this can be expanded to a struct including versioning or battery flags.

Command Mapping Contract:

Command ID (Hex)	Recognized Gesture (Prototype Set)	Track ID	Receiver Action
0x01	Advance (Fist)	0x01	Plays "Advance" audio track
0x02	Hold (Open Hand)	0x02	Plays "Hold Position" audio track
0x03	Retreat (Pointing)	0x03	Plays "Retreat" audio track
0x04	Cover (Two Fingers)	0x04	Plays "Take Cover" audio track
0x05	Confirm (Thumbs Up)	0x05	Plays "Affirmative" audio track
0xFF	Low Confidence / Error	None	Payload ignored, no audio played

8. Boundary Conditions & Error Handling

- **IMU Initialization Failure:** If the IMUSensor initialization fails during system startup, the system enters the Error state (LED blinks red continuously) and halts execution.
- **Low Confidence Recognition:** If the gesture confidence is < 75%, no payload is sent. The LED blinks Red twice, and the system resets to Idle.
- **Movement Timeout:** If the GloveController enters the Collecting state but the buffer is not filled within 3 seconds, the state machine aborts the collection and returns to the Idle state to conserve power.
- **Low Battery Voltage:** If the internal ADC reads a battery voltage dropping below 3.3V, the system enters the Low Battery state. The RGB LED blinks Red continuously, and the ML inference cycle is suspended to prevent erratic sensor readings.

9. Glossary

- **ADC (Analog-to-Digital Converter):** A hardware peripheral on the ESP32 used to convert the continuous physical voltage from the flex sensors into a discrete 12-bit digital number.
- **DFPlayer Mini:** A compact MP3/audio module that decodes and plays pre-recorded audio files directly from a microSD card, controlled via UART.
- **Edge Computing:** The practice of processing data locally on the device (the ESP32-WROOM-32) rather than relying on a centralized cloud, ensuring zero-latency and offline operation.
- **ESP-NOW:** A connectionless, low-power Wi-Fi communication protocol developed by Espressif. It is used in this project to send rapid digital triggers from the glove to the Communication Unit.
- **HAL (Hardware Abstraction Layer):** A low-level software layer that hides the specific physical details of the hardware (e.g., I2C registers) from the higher-level application logic.
- **I2C (Inter-Integrated Circuit):** A synchronous, multi-controller serial communication bus used to interface the ESP32-WROOM-32 with the MPU6050 IMU sensor.
- **TinyML:** A subfield of machine learning focused on deploying models on resource-constrained microcontrollers like the ESP32.
- **UART (Universal Asynchronous Receiver-Transmitter):** A hardware communication protocol that uses asynchronous serial communication, utilized here to send playback commands to the audio module.

10. References

[1] A. Esin, B. Çakmak, Ö. E. Dikici, and Ş. Kurtulmuş, "AKKE - High-Level Design Report (SMART COMMAND AND CONTROL GLOVE)," TED University, CMPE 491 Senior Project I, Fall 2025.

[2] A. Esin, B. Çakmak, Ö. E. Dikici, and Ş. Kurtulmuş, "AKKE - Project Specifications Report," TED University, CMPE 491 Senior Project I, Fall 2025.

[3] A. Esin, B. Çakmak, Ö. E. Dikici, and Ş. Kurtulmuş, "AKKE - Analysis Report," TED University, CMPE 491 Senior Project I, Fall 2025.

[4] B. Bruegge and A. H. Dutoit, *Object-Oriented Software Engineering: Using UML, Patterns, and Java*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2004.

[5] Espressif Systems, "ESP32-WROOM-32 Datasheet," ver. 3.3. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32_datasheet_en.pdf

[6] InvenSense, "MPU-6000 and MPU-6050 Product Specification," Revision 3.4. [Online].

[7] Espressif Systems, "ESP-NOW User Guide," ESP-IDF Programming Guide. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_now.html